NFT 마켓 학습 가이드

TokenizeArt(ERC-721)와 AMM 안의 고정가 마켓플레이스를
그림과 숫자로, 컨트랙트부터 프론트엔드까지

대상 파일: `code/src/TokenizeArt.sol`, `code/src/AMM.sol`(마켓 부분), `code/test/NFTMarket.t.sol`, `frontend/src/App.tsx`(MyNFTs · ListNFTForm · Market · ListingCard)
함께 보는 라이브러리: OpenZeppelin Contracts 5.7.0 (`ERC721`), wagmi 3 · viem 2
이 문서의 숫자, 이벤트, 가스, 에러 선택자는 실제 anvil 실행과 Foundry 테스트로 확인한 값입니다.

목차

1. NFT란: ERC-20과 무엇이 다른가
2. OpenZeppelin ERC721 내부: 장부 4개와 허락 3가지
3. TokenizeArt.sol 한 줄씩
4. 메타데이터와 IPFS: 이미지는 어디에 있나
5. 마켓플레이스 설계: 에스크로, Listing, 상태 변화
6. listNFT: 판매 등록
7. cancelListing: 판매 취소 (storage vs memory)
8. buyNFT: 구매 (실제 실행 기록)
9. 이벤트: 서브그래프가 읽을 기록
10. 테스트 8개와 추가로 검증한 사실
11. 프론트엔드: 화면이 컨트랙트와 대화하는 법
12. 전체 시나리오 추적표
13. 보안 메모, 알려진 한계, 개선점
14. 스스로 점검하기 (문제와 답)
15. 부록: 선택자·토픽 표, 한 장 요약

읽는 법

각 장은 **개념** → **그림** → **코드** → **한 줄씩 해설** → **숫자** 순서입니다. 보라색 상자는 **문법 메모**, 주황 상자는 **주의**, 초록 상자는 **핵심 정리**입니다. ERC-20 가이드(`erc20-guide.pdf`)를 먼저 읽었다면 비교하면서 읽으면 훨씬 쉽습니다.

CHAPTER 1

NFT란: ERC-20과 무엇이 다른가

NFT(Non-Fungible Token, 대체 불가능 토큰)는 **번호마다 서로 다른** 토큰입니다. 표준 이름은 **ERC-721**입니다. ERC-20의 1 SW42와 다른 1 SW42는 완전히 같지만, NFT #0과 NFT #1은 서로 다른 물건입니다. 콘서트 티켓의 좌석 번호와 비슷합니다.

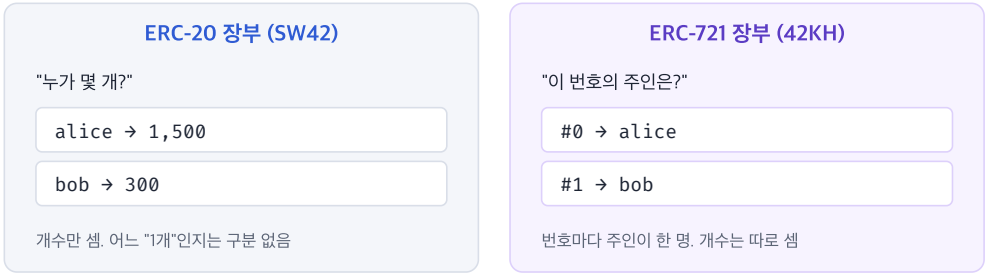


그림 1. ERC-20은 "주소 → 개수", ERC-721은 "번호 → 주인"이 중심입니다.

1-1. 한눈에 비교

	ERC-20 (SW42, R2B)	ERC-721 (42KH)
핵심 장부	`balanceOf[주소] = 개수`	`ownerOf[번호] = 주소`
단위	wei (10^{18}으로 나눠 표시)	개수는 정수 그대로, 소수점 없음
보내기	`transfer(to, 100)` : 100**개**	`transferFrom(from, to, 0)` : **#0번**
허락	`approve(spender, 100)` : 100개까지	`approve(to, 0)` : #0번을
허락 확인	`allowance(owner, spender)` → 숫자	`getApproved(0)` → **주소**
전부 허락	무제한 = `type(uint256).max`	`setApprovalForAll(operator, true)`
정보	`name` , `symbol` , `decimals`	`name` , `symbol` , `tokenURI(번호)`

재미있는 사실: 같은 선택자, 다른 뜻

ERC-20의 `approve(address,uint256)` 와 ERC-721의 `approve(address,uint256)` 는 시그니처가 같아서 **함수 선택자도 `0x095ea7b3` 로 똑같습니다**. EVM 입장에서는 같은 호출입니다. 두 번째 숫자를 "개수"로 읽을지 "번호"로 읽을지는 **받는 컨트랙트가 정합니다**. 이벤트 `Transfer` 와 `Approval` 의 topic0도 두 표준이 같습니다(9장).

1-2. 우리 프로젝트의 NFT

항목	값
컨트랙트	`TokenizeArt` (`code/src/TokenizeArt.sol`)
`name()` / `symbol()`	42 Korean Hacker / 42KH
발행	`mint(to)` : owner(배포자)만. 번호는 0부터 1씩 증가
메타데이터	모든 번호가 같은 `ipfs://bafkrei…vd6i`
배포 직후	배포 스크립트가 #0, #1, #2를 배포자에게 발행
마켓	별도 컨트랙트가 아니라 **AMM 컨트랙트 안**의 함수 3개: `listNFT` , `cancelListing` , `buyNFT`

과제 요구사항은 "고정 가격, Token A 또는 B로 결제, NFT는 AMM에 맡겨 둠(escrow), 판매자는 취소 가능, 풀 로직과 독립"입니다. 이 문서는 이 요구사항이 코드의 어디에서 어떻게 지켜지는지를 따라갑니다.

CHAPTER 2

OpenZeppelin ERC721 내부: 장부 4개와 허락 3가지

TokenizeArt는 `ERC721`을 상속합니다. 표준 기능은 전부 여기서 옵니다. 위치: `lib/openzeppelin-contracts/contracts/token/ERC721/ERC721.sol`.

2-1. 장부 4개

```
mapping(uint256 tokenId => address) private _owners;                              // 번호 → 주인
mapping(address owner => uint256) private _balances;                              // 주인 → 가진 개수
mapping(uint256 tokenId => address) private _tokenApprovals;                      // 번호 → 허락받은 1명
mapping(address owner => mapping(address operator => bool)) private _operatorApprovals; // 주인 → (대리인 → 전부 허락?)
```

장부	배포 직후 값 (예)	읽는 함수
`_owners`	#0, #1, #2 → 배포자	`ownerOf(id)`
`_balances`	배포자 → 3	`balanceOf(owner)`
`_tokenApprovals`	모두 0x0	`getApproved(id)`
`_operatorApprovals`	모두 false	`isApprovedForAll(owner, op)`

"내가 가진 번호 목록"은 없습니다

`balanceOf(나) = 3`은 알 수 있지만, **그게 몇 번인지**는 장부에 없습니다. `_owners`는 "번호 → 주인" 방향뿐이기 때문입니다. 목록이 필요하면 `ERC721Enumerable` 확장을 쓰거나, 이벤트를 모아야 합니다(서브그래프). 그래서 프론트는 0~19번에 `ownerOf`를 하나씩 묻습니다(11장).

2-2. 존재하지 않는 번호: `_requireOwned`

```
function ownerOf(uint256 tokenId) public view virtual returns (address) {
    return _requireOwned(tokenId);   // 주인이 0x0이면 revert ERC721NonexistentToken(tokenId)
}
```

anvil에서 `ownerOf(3)`을 부르면 `custom error 0x7e273289`가 나옵니다. 이것이 `ERC721NonexistentToken(uint256)`의 선택자입니다. ERC-20의 `balanceOf`는 모르는 주소에 0을 돌려주지만, ERC-721의 `ownerOf`는 없는 번호에 **에러**를 냅니다.

2-3. 허락 3가지: `_isAuthorized`

```
function _isAuthorized(address owner, address spender, uint256 tokenId) internal view virtual returns (bool) {
    return spender != address(0) &&
        (owner == spender                          // ① 주인 본인
         || isApprovedForAll(owner, spender)       // ② 전부 허락받은 대리인
         || _getApproved(tokenId) == spender);     // ③ 이 번호만 허락받은 사람
}
```

방법	설정 함수	범위	우리 프론트
① 주인 본인	(설정 없음)	자기 NFT	
② operator	`setApprovalForAll(op, true)`	그 주인의 **모든** NFT	사용 안 함
③ 번호별 허락	`approve(to, id)`	**그 번호 하나**, 동시에 1명	**사용** (ListNFTForm)

2-4. `_update` : 모든 이동의 단일 통로

```
function _update(address to, uint256 tokenId, address auth) internal virtual returns (address) {
    address from = _ownerOf(tokenId);
    if (auth != address(0)) {
        _checkAuthorized(from, auth, tokenId);       // 허락 확인 (위의 3가지)
    }
    if (from != address(0)) {
        _approve(address(0), tokenId, address(0), false);   // ★ 번호별 허락 초기화
        unchecked { _balances[from] -= 1; }
    }
    if (to != address(0)) {
        unchecked { _balances[to] += 1; }
    }
    _owners[tokenId] = to;
    emit Transfer(from, to, tokenId);
    return from;
}
```

★ 이동하면 허락이 자동으로 지워집니다

NFT가 움직일 때마다 `_tokenApprovals[tokenId]` 가 0x0으로 초기화됩니다. 새 주인에게 이전 주인이 준 허락이 남아 있으면 안 되기 때문입니다. 그래서 판매 등록 후 `getApproved(0)` 이 AMM에서 **0x0**으로 돌아갑니다(실제 anvil 값). ERC-20은 쓴 만큼 **차감**, ERC-721은 이동하면 **삭제**입니다.

2-5. transferFrom과 safeTransferFrom

```
function transferFrom(address from, address to, uint256 tokenId) public virtual {
    if (to == address(0)) revert ERC721InvalidReceiver(address(0));
    address previousOwner = _update(to, tokenId, _msgSender());    // auth = 호출자
    if (previousOwner != from) revert ERC721IncorrectOwner(from, tokenId, previousOwner);
}
function safeTransferFrom(address from, address to, uint256 tokenId, bytes memory data) public virtual {
    transferFrom(from, to, tokenId);
    ERC721Utils.checkOnERC721Received(_msgSender(), from, to, tokenId, data);   // 받는 쪽 확인
}
```

`safe` 버전은 받는 주소가 **컨트랙트**일 때 그 컨트랙트에 `onERC721Received` 를 물어봅니다. NFT를 다룰 줄 모르는 컨트랙트로 보내면 NFT가 영원히 갇히기 때문입니다. 받는 쪽이 일반 지갑(EOA)이면 확인을 건너뜁니다.

에러	선택자	언제
`ERC721NonexistentToken(id)`	`0x7e273289`	없는 번호
`ERC721InsufficientApproval(spender, id)`	`0x177e802f`	허락 없이 옮기려 함
`ERC721IncorrectOwner(from, id, owner)`	`0x64283d7b`	`from` 이 실제 주인이 아님
`ERC721InvalidReceiver(to)`		0x0으로 보내거나, 받을 수 없는 컨트랙트

CHAPTER 3

TokenizeArt.sol 한 줄씩

```
contract TokenizeArt is ERC721, Ownable {
    uint256 private _nextTokenId;      // 다음에 민팅될 NFT의 ID
    string private _tokenURI;          // 모든 NFT가 참조하는 메타데이터 URI

    constructor(string memory metadataURI) ERC721("42 Korean Hacker", "42KH") Ownable(msg.sender) {
        _tokenURI = metadataURI;
    }

    function mint(address to) public onlyOwner {
        uint256 tokenId = _nextTokenId;
        _nextTokenId++;
        _safeMint(to, tokenId);
    }

    function tokenURI(uint256 tokenId) public view override returns (string memory) {
        _requireOwned(tokenId);
        return _tokenURI;
    }
}
```

3-1. 상태 변수

변수	뜻	주의
`_nextTokenId`	다음 발행 번호. 0에서 시작	`private`이라 밖에서 못 읽음 → 프론트는 "총 몇 개"를 알 수 없음
`_tokenURI`	메타데이터 주소 (문자열 하나)	모든 번호가 공유

문법 메모: string과 memory

`string`은 길이가 정해지지 않은 참조 타입이라 위치를 적어야 합니다. 생성자 인자 `string memory metadataURI`는 "임시 메모리에 있는 문자열"이고, 상태 변수 `_tokenURI`에 대입하는 순간 **storage로 복사**됩니다. 반환 타입의 `string memory`도 "복사본을 돌려준다"는 뜻입니다.

3-2. constructor

- `ERC721("42 Korean Hacker", "42KH")` : 이름과 심볼 설정
- `Ownable(msg.sender)` : 배포자가 owner
- `_tokenURI = metadataURI` : 배포 스크립트가 넘긴 `ipfs://bafkrei…vd6i` 저장

3-3. mint(to)

줄	설명
`onlyOwner`	owner만 발행 (ERC-20 가이드 9장의 modifier)
`uint256 tokenId = _nextTokenId;`	지금 번호를 먼저 꺼냄
`_nextTokenId++;`	다음 번호 준비. `++`는 1 더하기
`_safeMint(to, tokenId);`	발행. `to`가 컨트랙트면 `onERC721Received` 확인 (2-5장)

배포 스크립트는 `for (i = 0; i < 3; i++) nft.mint(deployer);`로 세 번 호출합니다.

호출	tokenId	호출 후 _nextTokenId	이벤트
1번째	0	1	`Transfer(0x0, 배포자, 0)`
2번째	1	2	`Transfer(0x0, 배포자, 1)`
3번째	2	3	`Transfer(0x0, 배포자, 2)`

3-4. tokenURI(tokenId): override

OpenZeppelin 기본 구현은 `baseURI + 번호`(예: `ipfs://폴더/0`, `ipfs://폴더/1`)를 돌려줍니다. 번호마다 다른 메타데이터를 가리키는 방식입니다. TokenizeArt는 이를 `override`해서 **번호와 상관없이 같은 URI**를 돌려줍니다. 그래서 #0, #1, #2가 모두 같은 그림입니다.

> **문법 메모: override와 _requireOwned**
>
> 부모의 `virtual` 함수를 자식이 바꿔 쓸 때 `override`를 붙입니다. `_requireOwned(tokenId)`를 먼저 부르는 이유는 "없는 번호의 메타데이터"를 돌려주지 않기 위해서입니다. 표준이 요구하는 동작입니다.

CHAPTER 4

메타데이터와 IPFS: 이미지는 어디에 있나

컨트랙트에는 **이미지가 없습니다**. 주소 문자열 하나만 있습니다. 이미지(1.6MB png)를 블록체인 storage에 넣으면 가스비가 감당할 수 없을 만큼 비싸기 때문입니다.

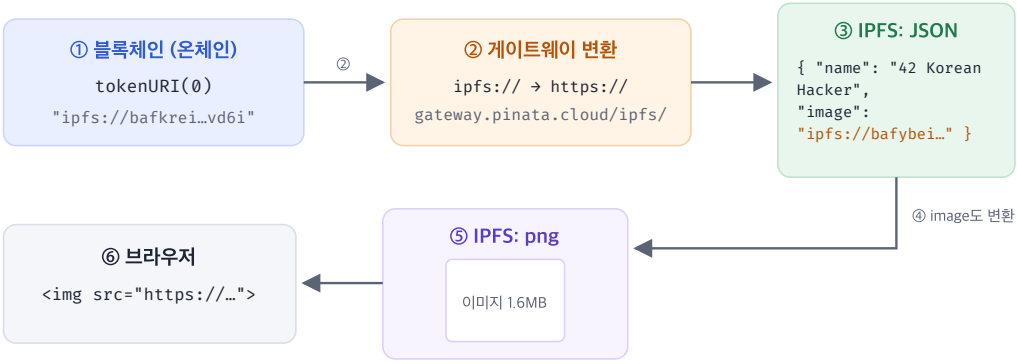


그림 2. 이미지까지 가는 길. 온체인에는 주소만 있고, 내용은 IPFS에 있습니다. CID가 두 번 등장합니다(JSON과 png).

4-1. CID: 내용으로 만든 주소

용어	뜻
CID (`bafkrei…`)	파일 **내용의 해시**. 내용이 1바이트만 바뀌어도 CID가 바뀜 → 같은 CID면 내용이 절대 바뀌지 않음
`ipfs://CID`	"IPFS에서 이 내용을 찾아라". 브라우저는 직접 못 엶
게이트웨이	CID를 찾아 일반 https로 내려주는 서버 (pinata, ipfs.io …). 어느 게이트웨이든 같은 파일
pin	누군가 그 파일을 계속 보관해 주는 것. 아무도 pin하지 않으면 사라질 수 있음

왜 해시 주소가 중요한가

일반 https 주소(`https://내서버/0.json`)라면 서버 주인이 나중에 그림을 바꿔치기할 수 있습니다. CID는 내용 자체의 지문이라서, 컨트랙트에 적힌 CID가 가리키는 내용은 **누구도 바꿀 수 없습니다**.

4-2. 실제 메타데이터

```
{
  "name": "42 Korean Hacker",
  "description": "NFT created for TokenizeArt project at 42 Lausanne.",
  "image": "ipfs://bafybeig3kruz6rm4si4nn5g4lpgiybvi6poxadtstjztm7vkvyeq4qp2oe",
  "artist": "sun-wkim",
  "attributes": [
    { "trait_type": "School", "value": "42 Lausanne" },
    { "trait_type": "Origin", "value": "Korea" },
    { "trait_type": "Type",   "value": "Hacker" }
  ]
}
```

`name` , `description` , `image` , `attributes` 는 OpenSea 등이 쓰는 **관례적인 형식**입니다. 지갑과 마켓이 이 이름으로 읽습니다.

4-3. 우리가 겪은 문제 두 가지

문제	원인	해결
이미지가 안 나옴	`replace('ipfs://', 'https://ipfs.io/ipfs/' + CID)` 로 **CID를 붙여 넣어** CID가 두 번 들어감	앞부분만 교체: `replace('ipfs://', 'https://gateway.pinata.cloud/ipfs/')`
URL이 맞아도 실패	`ipfs.io`, `dweb.link` 가 일반 요청에 429 응답 (service worker 전용으로 전환)	`gateway.pinata.cloud` 사용 (메타데이터, 이미지 모두 200 확인)

CHAPTER 5

마켓플레이스 설계: 에스크로, Listing, 상태 변화

5-1. 에스크로(escrow)란

에스크로는 **믿을 수 있는 제3자가 물건을 맡아 두는 방식**입니다. 중고거래 안전결제와 같습니다. 판매자가 NFT를 **AMM 컨트랙트에 맡기면**, 판매 중에는 판매자도 그 NFT를 다른 데 팔거나 옮길 수 없습니다. 구매자가 돈을 내는 순간 컨트랙트가 NFT를 구매자에게 넘깁니다. 사람을 믿을 필요가 없고 **코드를 믿으면** 됩니다.

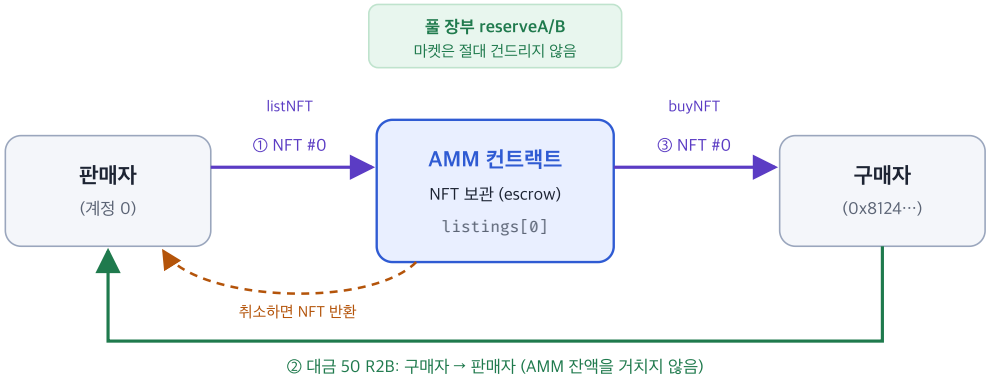


그림 3. 에스크로 흐름. NFT는 AMM을 거쳐 가고, 대금은 구매자에서 판매자에게 바로 갑니다.

5-2. Listing struct

```
struct Listing {
    address seller;        // [0] 누가 등록했나 (대금 수령, 취소 권한)
    IERC721 nft;           // [1] 어느 NFT 컨트랙트
    uint256 tokenId;       // [2] 그 안의 몇 번
    uint256 price;         // [3] 가격 (결제 토큰 wei 단위)
    IERC20 paymentToken;   // [4] tokenA 또는 tokenB
    bool active;           // [5] 판매 중인가 (팔리거나 취소되면 false)
}

uint256 public nextListingId;                    // 다음 판매 번호. 0부터, 늘기만 함
mapping(uint256 => Listing) public listings;     // 판매 번호 → 판매 정보
```

문법 메모: struct, 그리고 IERC721 타입

`struct` 는 여러 값을 묶은 **사용자 정의 타입**입니다. `IERC721 nft` 는 저장 형태로는 주소와 같지만, `listing.nft.transferFrom(...)` 처럼 **바로 함수를 부를 수 있는 타입**입니다. `IERC721(nft)` 는 주소를 이 타입으로 바꾸는 형변환입니다.

public struct getter는 "튜플"을 돌려줍니다

`listings(0)` 을 부르면 struct가 아니라 **필드 6개를 순서대로 담은 튜플**이 옵니다. 반환값은 이름이 아니라 **순서로 해석**됩니다. 그래서 프론트 ABI의 반환 순서가 struct와 한 칸이라도 다르면 값이 엉뚱하게 읽힙니다. 우리가 실제로 겪은 일입니다(11장).

[0]	[1]	[2]	[3]	[4]	[5]
seller	nft	tokenId	price	paymentToken	active
0xf39F…	0xCf7E…	0	50e18	0xe7f1… (R2B)	true

5-3. 판매 건의 일생 (상태 변화)

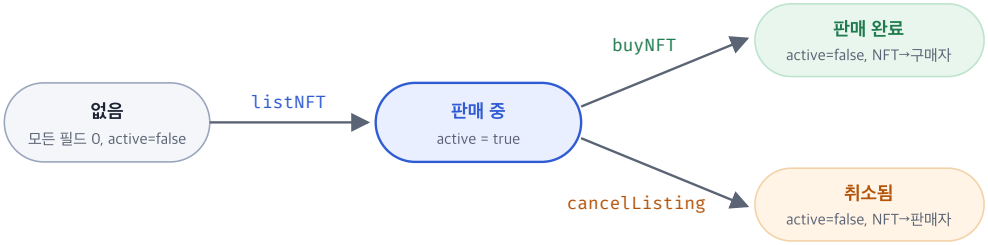


그림 4. 한 판매 건은 "판매 중"을 거쳐 "판매 완료" 또는 "취소됨"으로 끝납니다. 끝난 건은 다시 판매 중으로 돌아가지 않습니다.

- 끝난 건의 데이터는 **지우지 않고** `active = false`로만 바꿉니다. 기록이 남습니다.
- 같은 NFT를 다시 팔면 **새 번호**를 받습니다. 취소 후 재등록하면 0번이 아니라 1번이 됩니다(Foundry로 확인).
- `nextListingId`는 "지금까지 등록된 총 건수"이지 "판매 중인 건수"가 아닙니다.

CHAPTER 6

listNFT: 판매 등록

```
function listNFT(address nft, uint256 tokenId, uint256 price, address paymentToken)
    external nonReentrant returns (uint256 listingId)
{
    // Checks
    require(paymentToken == address(tokenA) || paymentToken == address(tokenB), "AMM: invalid payment token");
    require(price > 0, "AMM: zero price");

    // Effects: 지금 번호에 저장하고, 다음 번호로
    listingId = nextListingId;
    listings[listingId] = Listing({
        seller: msg.sender,
        nft: IERC721(nft),
        tokenId: tokenId,
        price: price,
        paymentToken: IERC20(paymentToken),
        active: true
    });
    nextListingId = nextListingId + 1;

    // Interactions: NFT를 판매자 → AMM으로 (escrow)
    IERC721(nft).transferFrom(msg.sender, address(this), tokenId);

    emit NFTListed(listingId, msg.sender, nft, tokenId, paymentToken, price);
}
```

6-1. 한 줄씩

줄	설명
`returns (uint256 listingId)`	**이름 붙은 반환값**. 함수 안에서 `listingId = …`로 대입하면 끝날 때 자동으로 반환됨 (`return` 생략 가능)
`require(paymentToken == tokenA || … tokenB)`	결제 토큰은 풀의 두 토큰만 허용 (과제 요구사항)
`require(price > 0)`	0원 판매 방지 (실수 방지)
`listingId = nextListingId;`	지금 번호(예: 0)를 이 판매 건의 번호로
`Listing({ seller: …, … })`	struct를 **이름 붙여** 만드는 문법. 순서를 틀릴 걱정이 없음
`seller: msg.sender`	호출한 사람이 판매자. 인자로 받지 않으므로 남을 판매자로 적을 수 없음
`nextListingId = nextListingId + 1;`	다음 판매는 1번
`IERC721(nft).transferFrom(msg.sender, address(this), tokenId)`	NFT를 AMM으로. `address(this)` = 이 컨트랙트(AMM) 자신의 주소
`emit NFTListed(…)`	서브그래프와 프론트가 읽을 기록

6-2. transferFrom이 지켜 주는 것 두 가지

listNFT 자체에는 "호출자가 NFT 주인인가?"를 확인하는 코드가 없습니다. 그 일을 **NFT 컨트랙트의 transferFrom이 대신** 해 줍니다(2-5장). NFT 컨트랙트 입장에서 `msg.sender`는 AMM이고, `from`은 listNFT를 부른 사람입니다.

상황 (Foundry로 확인)	막는 곳	에러
alice가 approve 없이 등록	`_checkAuthorized` : AMM이 허락받지 않음	`ERC721InsufficientApproval` (`0x177e802f`)
alice가 approve해 둔 #0을 **bob이** 등록 시도	`previousOwner(alice) != from(bob)`	`ERC721IncorrectOwner` (`0x64283d7b`)

두 번째가 중요한 이유

alice가 approve만 해 두고 아직 등록하지 않은 순간, AMM은 #0을 옮길 권한이 있습니다. 만약 `from` 확인이 없다면 bob이 "#0을 내 이름(seller = bob)으로 등록"해서 판매 대금을 가로챌 수 있습니다. listNFT가 `from`에 `msg.sender`를 넣기 때문에, 주인이 아닌 사람의 호출은 NFT 컨트랙트에서 거부됩니다.

6-3. 왜 safeTransferFrom이 아니라 transferFrom인가

NatSpec에 이렇게 적어 두었습니다: *"this contract is the receiver and is known to handle ERC-721, so the onERC721Received check of safeTransferFrom is not needed."* 받는 쪽이 **AMM 자신**이고, AMM은 NFT를 다룰 줄 압니다(cancel/buy로 내보낼 수 있음). 그래서 받는 쪽 확인이 필요 없습니다. 만약 `safeTransferFrom`을 쓰면 AMM이 `onERC721Received`를 구현해야 하고, 구현하지 않으면 등록이 실패합니다.

6-4. CEI 순서와 nonReentrant

장부(`listings`, `nextListingId`)를 먼저 쓰고, 외부 호출(`transferFrom`)을 마지막에 합니다. `nft` 주소는 **호출자가 아무 주소나** 넣을 수 있으므로, 악의적인 NFT 컨트랙트가 `transferFrom` 안에서 AMM을 다시 호출(재진입)할 수 있습니다. `nonReentrant`가 그런 재진입을 막고, CEI 순서가 한 번 더 안전하게 합니다.

6-5. 숫자로: #0을 50 R2B에 등록 (실제 anvil)

항목	전	후
`nft.getApproved(0)`	AMM (approve 직후)	**0x0** (이동하면서 초기화)
`nft.ownerOf(0)`	계정 0	**AMM**
`nft.balanceOf(계정 0)`	3	2
`amm.nextListingId()`	0	1
`amm.listings(0)`	전부 0	(계정 0, NFT, 0, 50e18, R2B, true)

가스: 약 18만 (Foundry gas report의 listNFT 중앙값 180,676). storage 칸 여러 개를 새로 쓰기 때문에 마켓 함수 중 가장 비쌉니다.

CHAPTER 7

cancelListing: 판매 취소

```
function cancelListing(uint256 listingId) external nonReentrant {
    Listing storage listing = listings[listingId];    // 원본을 가리킴

    // Checks
    require(listing.active, "AMM: listing not active");
    require(listing.seller == msg.sender, "AMM: not the seller");

    // Effects
    listing.active = false;

    // Interactions: AMM → 판매자로 NFT 반환
    listing.nft.transferFrom(address(this), msg.sender, listing.tokenId);

    emit NFTListingCancelled(listingId, msg.sender);
}
```

7-1. storage vs memory: 우리가 실제로 고친 버그

처음에는 `Listing memory listing = listings[listingId];`로 썼다가 `storage`로 고쳤습니다. 이 한 단어가 결정적입니다.

	`Listing storage listing`	`Listing memory listing`
무엇인가	storage에 있는 **원본을 가리키는 참조**	원본을 메모리로 **복사**한 것
`listing.active = false`	원본이 바뀜 ✅	복사본만 바뀌고, 함수가 끝나면 사라짐 ❌
결과	취소가 기록됨	NFT는 돌려줬는데 장부는 여전히 "판매 중"인 **유령 판매 건**이 남음

유령 판매 건은 어떤 문제를 일으킬까요?

- 누가 사려고 하면 대금 이동 후 NFT 이동(`AMM → 구매자`)에서 실패합니다. AMM이 그 NFT를 더 이상 갖고 있지 않기 때문입니다. tx 전체가 되돌아가므로 **돈을 잃지는 않지만**, 화면에는 영원히 "판매 중"으로 보입니다.
- **더 위험한 경우**: 판매자가 같은 NFT를 70 R2B로 **다시 등록**(새 1번)하면 NFT가 다시 AMM에 들어옵니다. 이때 누군가 유령 0번(50 R2B)을 사면 NFT 이동이 **성공**합니다. 판매자가 원한 70이 아니라 **옛 가격 50**에 팔려 버리고, 1번은 NFT 없는 판매 건이 됩니다. `storage`를 `memory`로 바꾼 임시 복사본으로 Foundry에서 재현해 확인했습니다(구매자 50 지불, NFT 획득, 1번은 active로 남음).

그래서 테스트 메시지가 이렇습니다

`assertFalse(_isActive(listingId), "listing must be inactive (storage, not memory)");` 이 테스트(`test_Cancel_ReturnsNFTAndDeactivates`)는 바로 이 버그를 다시 만들지 않도록 지켜 줍니다.

비유

`storage`는 공유 문서의 **원본 링크**, `memory`는 그 문서를 **다운로드한 사본**입니다. 사본을 아무리 고쳐도 원본은 그대로입니다.

7-2. 검사 두 개의 순서

검사	막는 것	테스트
`listing.active`	이미 팔렸거나 취소된 건을 다시 취소 (없는 NFT를 반환하려 함)	
`listing.seller == msg.sender`	남의 판매 건 취소	`test_RevertWhen_NonSellerCancels`

취소에는 approve가 필요 없습니다. NFT를 옮기는 주체가 AMM이고, AMM은 그 NFT의 **현재 주인**이기 때문입니다(2-3장의 ① 주인 본인).

7-3. 숫자로

항목	취소 전	취소 후
`ownerOf(tokenId)`	AMM	판매자
`listings(id).active`	true	false
그 외 필드	그대로 남음 (기록)	

가스: 약 5만~7만 (gas report 53,174~74,666).

CHAPTER 8

buyNFT: 구매

```
function buyNFT(uint256 listingId) external nonReentrant {
    Listing storage listing = listings[listingId];

    // Checks
    require(listing.active, "AMM: listing not active");

    // Effects
    listing.active = false;

    // Interactions
    listing.paymentToken.safeTransferFrom(msg.sender, listing.seller, listing.price);  // 대금: 구매자 → 판매자
    listing.nft.transferFrom(address(this), msg.sender, listing.tokenId);              // NFT: AMM → 구매자

    emit NFTSold(listingId, msg.sender, listing.seller, listing.price);
}
```

8-1. 한 줄씩

줄	설명
`require(listing.active)`	판매 중인 건만. 두 번 팔기 방지 (`test_RevertWhen_BuyingSoldListing`)
`listing.active = false`	외부 호출 **전에** 먼저 닫음 (CEI). 재진입해도 두 번째는 `not active`로 막힘
`paymentToken.safeTransferFrom(구매자, 판매자, price)`	대금이 **판매자에게 바로**. AMM 잔액을 거치지 않음 → 풀과 독립. 구매자가 AMM에게 approve 해 둬야 함
`nft.transferFrom(AMM, 구매자, tokenId)`	맡아 둔 NFT를 구매자에게. AMM이 주인이라 approve 불필요
`emit NFTSold(…)`	판매 기록

"원자성": 둘 다 되거나, 둘 다 안 되거나

대금 이동과 NFT 이동은 **한 트랜잭션** 안에 있습니다. 구매자의 R2B가 모자라 대금 이동이 실패하면 `active = false`도, NFT 이동도 **전부 취소**됩니다. "돈만 내고 NFT를 못 받는" 상황이나 그 반대가 생길 수 없습니다.

8-2. 실제 실행 기록 (anvil에서 스냅샷을 찍고 실행한 뒤 되돌림)

구매자 `0x8124…9619`가 ① R2B faucet → ② `R2B.approve(AMM, 50e18)` → ③ `buyNFT(0)`을 실행했습니다.

항목	구매 전	구매 후
`nft.ownerOf(0)`	AMM	**구매자**
구매자 R2B	1,000	**950**
판매자(계정 0) R2B	960,000	**960,050**
AMM의 R2B 잔액	40,000	**40,000** (그대로)
`reserveB`	40,000	**40,000** (그대로)
`listings(0).active`	true	**false**
tx status / 가스	성공 (0x1) / **95,395** gas	

같은 건을 계정 0이 다시 사려고 하면 `"AMM: listing not active"`로 실패했습니다.

이 tx에서 나온 로그 3개 (순서대로)

#	컨트랙트	이벤트	indexed (topics)	data
0	R2B	`Transfer` (ERC-20)	from = 구매자, to = 판매자	50e18
1	NFT	`Transfer` (ERC-721)	from = AMM, to = 구매자, **tokenId = 0**	(비어 있음)
2	AMM	`NFTSold`	listingId = 0, buyer, seller	price = 50e18

순서가 코드 순서와 정확히 같습니다(대금 → NFT → 이벤트).

CHAPTER 9

이벤트: 서브그래프가 읽을 기록

```
event NFTListed(uint256 indexed listingId, address indexed seller, address indexed nft,
                uint256 tokenId, address paymentToken, uint256 price);
event NFTListingCancelled(uint256 indexed listingId, address indexed seller);
event NFTSold(uint256 indexed listingId, address indexed buyer, address indexed seller, uint256 price);
```

이벤트	topic0 (앞부분)	indexed	data
`NFTListed`	`0x1e25fc8c…`	listingId, seller, nft	tokenId, paymentToken, price
`NFTListingCancelled`	`0x60d684fb…`	listingId, seller	(없음)
`NFTSold`	`0x8562eb94…`	listingId, buyer, seller	price

9-1. 왜 이 값들을 indexed로 했나

- `listingId` : "3번 판매 건의 기록만"처럼 한 건을 추적
- `seller` / `buyer` : "내가 판 것, 내가 산 것" 조회
- `nft` : 어느 NFT 컬렉션인지로 필터
- indexed는 이벤트당 최대 3개라서, 가격 같은 값은 data에 둡니다.

9-2. ERC-20 Transfer와 ERC-721 Transfer 구분하기

두 표준의 `Transfer(address,address,uint256)`은 시그니처가 같아서 **topic0도 `0xddf252ad…`로 똑같습니다**. 차이는 **세 번째 값이 indexed인가**입니다. 8장의 실제 로그를 보면 바로 보입니다.

	ERC-20 Transfer (로그 #0)	ERC-721 Transfer (로그 #1)
topics 개수	3 (topic0, from, to)	**4** (topic0, from, to, tokenId)
data	value (50e18)	**비어 있음**

서브그래프를 만들 때 이벤트를 **컨트랙트 주소별로** 등록하기 때문에 섞이지는 않지만, 블록 탐색기에서 로그를 읽을 때 알아 두면 좋습니다.

9-3. 서브그래프에서 할 일 미리보기

이벤트	서브그래프에서
`NFTListed`	Listing 엔티티 생성 (status = ACTIVE)
`NFTListingCancelled`	그 Listing의 status = CANCELLED
`NFTSold`	status = SOLD, buyer 기록
NFT의 `Transfer`	"주소 → 가진 NFT 목록" 유지 → 프론트의 0~19번 하나씩 묻기를 대체

CHAPTER 10

테스트 8개와 추가로 검증한 사실

`code/test/NFTMarket.t.sol`. 등장인물: alice(판매자, NFT #0 소유), bob과 carol(구매자, 각 1만 A·B). `PRICE = 500 ether`(= 500 토큰).

#	테스트	확인하는 것
A	`test_List_EscrowsNFTAndStoresListing`	첫 번호 0, `nextListingId` 1, NFT가 AMM에, 필드 저장, active
A	`test_List_EmitsEvent`	`NFTListed(0, alice, nft, 0, tokenA, 500e18)` 가 정확히 나옴 (`expectEmit`)
B	`test_Cancel_ReturnsNFTAndDeactivates`	NFT가 alice에게 돌아옴, active false (storage 버그 방지)
C	`test_Buy_PaysSellerAndTransfersNFT`	NFT → bob, alice +500, bob −500, **AMM 잔액 0**, active false
D	`test_RevertWhen_NonSellerCancels`	bob이 취소 → "AMM: not the seller"
E	`test_RevertWhen_BuyingSoldListing`	팔린 건을 carol이 사려 함 → "AMM: listing not active"
F	`test_RevertWhen_InvalidPaymentToken`	풀과 무관한 토큰으로 등록 → "AMM: invalid payment token"
G	`test_Marketplace_DoesNotTouchPoolReserves`	유동성이 있는 상태에서 등록+구매해도 `reserveA/B` 그대로

10-1. 테스트 문법 다시 보기

코드	뜻
`vm.startPrank(alice) … vm.stopPrank()`	그 사이의 모든 호출을 alice가 한 것으로
`vm.prank(bob)`	바로 다음 호출 **하나만** bob으로
`vm.expectRevert(bytes("AMM: not the seller"))`	다음 호출이 이 메시지로 실패해야 통과
`vm.expectEmit(true, true, true, true, address(amm))`	다음 호출이 아래 `emit`과 같은 이벤트를 내야 함 (topic1~3, data 모두 비교)
`(,,,,, active) = amm.listings(id)`	튜플 6개 중 마지막만 받기 (쉼표로 건너뜀)

expectEmit 전에 approve를 먼저 하는 이유

`test_List_EmitsEvent`는 `nft.approve`를 `expectEmit`보다 **먼저** 호출합니다. approve도 `Approval` 이벤트를 내므로, expectEmit 뒤에 두면 "다음 호출"이 approve가 되어 비교 대상이 꼬입니다.

10-2. 이 문서를 쓰면서 추가로 검증한 사실

임시 Foundry 테스트와 anvil 스냅샷으로 확인했습니다(테스트 파일은 삭제).

상황	결과
approve 없이 listNFT	❌ `ERC721InsufficientApproval` (`0x177e802f`)
alice가 approve한 #0을 bob이 listNFT	❌ `ERC721IncorrectOwner` (`0x64283d7b`)
등록 후 `getApproved(0)`	0x0 (자동 초기화)
등록 후 `balanceOf`	alice 2 → 1, AMM 0 → 1
취소 후 재등록	새 번호 1, `nextListingId` = 2
실제 구매 (anvil)	성공, 95,395 gas, 로그 3개 (8장)
팔린 건 재구매	❌ "AMM: listing not active"

CHAPTER 11

프론트엔드: 화면이 컨트랙트와 대화하는 법

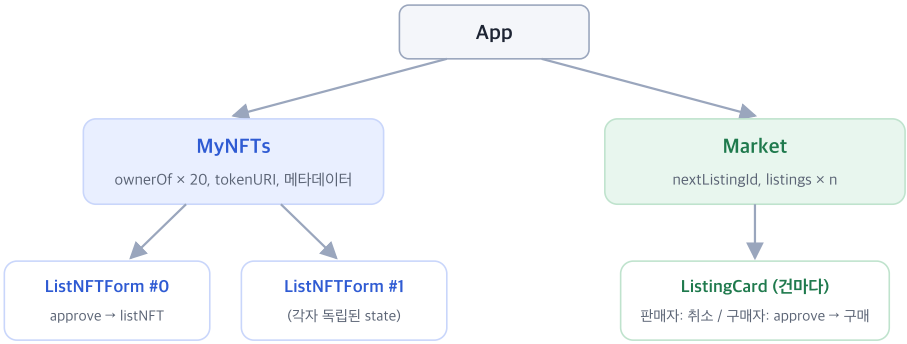


그림 5. NFT 관련 컴포넌트 구조. 부모는 목록을 읽고, 자식은 한 건씩 쓰기(tx)를 담당합니다.

11-1. MyNFTs: "내 NFT"를 찾는 법

```
const tokenIds = Array.from({ length: MAX_NFT_ID }, (_, i) => BigInt(i))   // [0n … 19n]
const owners = useReadContracts({
  contracts: tokenIds.map((id) => ({ address: addresses?.nft, abi: nftAbi, functionName: 'ownerOf', args: [id] }))
as const),
  allowFailure: true,            // 없는 번호는 revert → 실패도 결과로 받기
  query: { enabled: !!addresses },
})
const myIds = tokenIds.filter((_, i) => {
  const r = owners.data?.[i]
  return !!me && r?.status === 'success' && isAddressEqual(r.result, me)
})
```

`allowFailure`	data 모양	하나라도 실패하면
`false`	`[값, 값, …]`	전체가 에러
`true`	`[{ status: 'success', result }, { status: 'failure', error }, …]`	그 칸만 failure

2장에서 본 대로 ERC-721에는 "내 번호 목록"이 없어서 0~19번을 모두 물어봅니다. #3~#19는 `ERC721NonexistentToken`으로 실패하므로 `allowFailure: true`가 꼭 필요합니다. `isAddressEqual`은 대소문자가 다른 같은 주소(`0xABC` / `0xabc`)를 같다고 판단합니다.

메타데이터는 wagmi가 아니라 useQuery

```
const metadata = useQuery({
  queryKey: ['nft-metadata', uri.data],                // 캐시 이름표 (직접 만듦)
  queryFn: async () => {
    const res = await fetch(ipfsToHttp(uri.data!))     // 블록체인이 아닌 일반 HTTP
    return (await res.json()) as { name: string; image: string }
  },
  enabled: !!uri.data,
})
```

wagmi의 읽기 훅도 내부는 이 `useQuery`입니다. 블록체인이 아닌 요청이라 직접 쓴 것뿐입니다. 모든 NFT가 같은 URI라서 키가 같고, 요청도 **한 번**만 나갑니다.

11-2. ListNFTForm: props와 카드별 state

```
<ListNFTForm tokenId={id} />                                  // 부모 → 자식: props
function ListNFTForm({ tokenId }: { tokenId: bigint }) { … }      // 자식이 받음
```

props	state
부모가 정해서 넘김, 자식은 읽기만	컴포넌트가 스스로 가짐, `setX`로 변경
`tokenId`	`priceText`, `payWithA`

같은 컴포넌트를 세 번 그리면 **독립된 인스턴스 세 개**가 생깁니다. 각자 자기 입력칸과 자기 `useWriteContract`를 가지므로, #0의 approve가 #1에 영향을 주지 않습니다.

```
const approved = useReadContract({ address: addresses?.nft, abi: nftAbi, functionName: 'getApproved', args:
[tokenId], … })
const isApproved = !!addresses && !!approved.data && isAddressEqual(approved.data, addresses.amm)
// isApproved ? [판매 등록] listNFT(nft, tokenId, price, paymentToken) : [NFT Approve] approve(amm, tokenId)
```

`!!approved.data &&`를 먼저 쓰는 이유: 로딩 중엔 `undefined`라 비교할 수 없습니다. `&&`는 앞이 false면 뒤를 실행하지 않습니다(단락 평가).

11-3. Market과 ListingCard

```
const next = useReadContract({ …, functionName: 'nextListingId' })
const ids = Array.from({ length: Number(next.data ?? 0n) }, (_, i) => BigInt(i))   // bigint → number
const raw = useReadContracts({ contracts: ids.map((id) => ({ …, functionName: 'listings', args: [id] }) as const),
allowFailure: false })

// 튜플 → 이름 있는 객체 → 판매 중만
const active: Listing[] = (raw.data ?? [])
  .map((l, i) => ({ id: ids[i], seller: l[0], tokenId: l[2], price: l[3], paymentToken: l[4], active: l[5] }))
  .filter((l) => l.active)
```

문법 메모

코드	뜻
`Number(x)`	bigint → number. 건수처럼 작은 수에만 안전 (2^{53} 한계)
`type Listing = { … }`	객체 모양에 이름 붙이기 (TypeScript 타입 별칭)
`.map((l, i) => …)`	`i` 는 순번. `ids[i]` 로 판매 번호와 짝지음
`.filter(...)`	조건이 true인 것만 남김

ListingCard의 버튼 3갈래

```
isSeller(판매자 == 나) ?
  ├─ 예  → [판매 취소]  cancelListing(id)                      approve 없음 (AMM이 NFT 주인)
  └─ 아니오 → allowance(나, AMM) >= price ?   (결제 토큰 컨트랙트에 묻기)
            ├─ 아니오 → [R2B Approve]  paymentToken.approve(AMM, price)
            └─ 예     → [구매]         buyNFT(id)
```

동작	옮겨지는 것	누구 것	approve?
취소	NFT: AMM → 판매자	AMM이 맡아 둔 NFT	❌
구매 (대금)	R2B: 구매자 → 판매자	**구매자의** 토큰	✅ 결제 토큰에
구매 (NFT)	NFT: AMM → 구매자	AMM이 맡아 둔 NFT	❌

11-4. 우리가 실제로 겪은 실수 모음

실수	증상	교훈
`listings` ABI 반환 순서를 다르게 씀 (nft, tokenId, price, paymentToken, seller)	`tsc` : price 자리에 주소 타입, `l[5]` 없음	반환값은 **순서로** 해석된다. struct 순서 그대로
`buyNFT(uint256 listingId, uint256 maxPrice)`	인자 개수 에러. 그대로 보냈다면 선택자 `0x1d85bf03` → 컨트랙트에 없는 함수	ABI는 **배포된 컨트랙트**를 그대로 옮긴다. 실제 선택자는 `0x51ed8288`
`'function mint()'` (실제는 `mint(address to)`)	다른 선택자 → revert	인자 타입이 하나라도 다르면 다른 함수
`ipfsToHttp`에 CID를 고정으로 넣음	이미지가 안 나옴	앞부분(`ipfs://`)만 교체
컴포넌트를 만들고 `App`에 안 넣음	`'Market' is declared but never read`	정의 + 사용(`<Market />`)이 한 세트

TypeScript가 막아 준 것

ABI를 `parseAbi` 로 쓰면 반환 타입까지 추론되므로, ABI와 사용하는 코드가 어긋나면 **실행 전에** 빨간 줄이 뜹니다. 타입 검사가 없었다면 화면에 엉뚱한 값만 나오고 원인을 찾기 어려웠을 것입니다.

CHAPTER 12

전체 시나리오 추적표

배포부터 구매까지, NFT #0을 중심으로 장부가 어떻게 바뀌는지 따라갑니다. **D** = 배포자(계정 0, 판매자), **U** = 구매자(0x8124…). R2B는 토큰 개수입니다.

#	호출 (누가)	ownerOf(0)	getApproved(0)	next ListingId	listings(0) .active	U R2B	D R2B
0	배포 + `mint(D)` × 3 (D)	D	0x0	0	false (빈 칸)	0	960,000
1	`nft.approve(AMM, 0)` (D)	D	**AMM**	0	false	0	960,000
2	`listNFT(nft, 0, 50e18, R2B)` (D)	**AMM**	**0x0**	**1**	**true**	0	960,000
3	`R2B.faucet()` (U)	AMM	0x0	1	true	**1,000**	960,000
4	`buyNFT(0)` (U) *approve 없이*	❌ `ERC20InsufficientAllowance` → 아무것도 안 바뀜					
5	`R2B.approve(AMM, 50e18)` (U)	AMM	0x0	1	true	1,000	960,000
6	`buyNFT(0)` (U)	**U**	0x0	1	**false**	**950**	**960,050**
7	`buyNFT(0)` (D)	❌ "AMM: listing not active"					

D의 R2B가 960,000인 이유: 초기 발행 1,000,000 중 40,000을 배포 스크립트가 풀에 넣었기 때문입니다. 4번은 구매자가 approve를 건너뛴 경우의 예시입니다(대금 이동이 OpenZeppelin의 allowance 검사에서 막힘).

단계별 로그

#	이벤트
0	`Transfer(0x0, D, 0)`, `(…, 1)`, `(…, 2)` (NFT 발행)
1	`Approval(D, AMM, 0)` (NFT, tokenId indexed)
2	`Transfer(D, AMM, 0)` (NFT), `NFTListed(0, D, nft, 0, R2B, 50e18)`
3	`Transfer(0x0, U, 1000e18)`, `FaucetClaimed(U, 1000e18)`
5	`Approval(U, AMM, 50e18)` (R2B)
6	`Transfer(U, D, 50e18)` (R2B), `Transfer(AMM, U, 0)` (NFT), `NFTSold(0, U, D, 50e18)`

풀과 독립 확인

전 과정에서 AMM의 `reserveA`, `reserveB`는 한 번도 바뀌지 않습니다. 대금은 AMM 잔액을 거치지 않고, NFT 보관은 ERC-20 장부와 무관하기 때문입니다. 테스트 G(`test_Marketplace_DoesNotTouchPoolReserves`)와 실제 anvil 구매(`reserveB` 40,000 그대로) 모두 이를 확인했습니다.

CHAPTER 13

보안 메모, 알려진 한계, 개선점

13-1. 이미 막혀 있는 것

위협	막는 장치
남의 NFT를 내 이름으로 등록	NFT의 `transferFrom`이 `from` = 실제 주인인지 확인 (`ERC721IncorrectOwner`)
남의 판매 건 취소	`require(listing.seller == msg.sender)`
같은 건 두 번 팔기 / 팔린 건 취소	`require(listing.active)` + 외부 호출 전에 `active = false` (CEI)
악성 NFT 컨트랙트의 재진입	`nonReentrant` (세 함수 모두)
돈만 내고 NFT 못 받음	한 tx 안의 원자성: 하나라도 실패하면 전부 되돌림
임의 토큰으로 결제 요구	결제 토큰은 tokenA/tokenB만
마켓이 풀 가격에 영향	대금이 판매자에게 직행, reserve 장부를 건드리지 않음

13-2. 알려진 한계 (README에 적을 것)

한계	설명	개선 방법
아무 NFT 컨트랙트나 등록 가능	`nft` 인자에 제한이 없음 → 가짜 NFT 컨트랙트를 만들어 등록하면 구매자가 속을 수 있음. 우리 프론트는 우리 NFT 주소만 쓰지만 컨트랙트는 열려 있음	허용 목록(allowlist) 또는 프론트에서 nft 주소 표시·검증
가격 변경 불가	바꾸려면 취소 후 재등록 (새 번호)	`updatePrice(id, newPrice)` 추가
만료 없음	취소할 때까지 영원히 판매 중	`deadline` 필드
로열티·수수료 없음	창작자 몫(ERC-2981)이나 마켓 수수료 없음	과제 범위 밖
자기 판매 건 구매 가능	막는 코드가 없음 (자기에게 돈을 내고 NFT를 돌려받는 것과 같음, 실익 없음)	`require(msg.sender != seller)`
목록 조회 비용	프론트가 `listings(0..n-1)`을 전부 읽음 → 건수가 늘면 느려짐	서브그래프로 active만 조회
"내 NFT" 조회	0~19번 고정 스캔 → 20번 이상은 안 보임	서브그래프 (NFT Transfer 인덱싱) 또는 ERC721Enumerable
setApprovalForAll 사용자	프론트는 `getApproved`만 봐서, 전체 허락한 사용자에게도 Approve 버튼을 보여줌	`isApprovedForAll`도 확인

메타데이터의 중앙화 지점

CID 덕분에 내용은 바꿔치기할 수 없지만, 누군가 그 파일을 **pin하지 않으면** 언젠가 IPFS에서 사라질 수 있습니다. 게이트웨이가 막히는 문제도 실제로 겪었습니다(4장). NFT의 "그림"은 결국 오프체인 저장소에 의존합니다.

CHAPTER 14

스스로 점검하기

Q1. ERC-721에서 "내가 가진 NFT 번호 목록"을 바로 알 수 없는 이유는? (2장)

`_owners`가 "번호 → 주인" 방향뿐이고, "주인 → 번호들" 장부가 없기 때문입니다. 개수(`balanceOf`)만 알 수 있습니다.

Q2. `ownerOf(3)`은 무엇을 돌려주나? (2장)

값이 아니라 revert `ERC721NonexistentToken(3)` (`0x7e273289`). 그래서 프론트는 `allowFailure: true`가 필요합니다.

Q3. ERC-20 approve와 ERC-721 approve의 차이는? 선택자는? (1장)

둘째 인자가 개수 vs 번호. 선택자는 둘 다 `0x095ea7b3`로 같습니다.

Q4. 판매 등록 직후 `getApproved(0)`은? 왜? (2-4장)

0x0. `_update`가 NFT가 움직일 때 번호별 허락을 초기화합니다.

Q5. listNFT에 "호출자가 주인인가" 검사가 없는데도 남의 NFT를 등록할 수 없는 이유는? (6-2장)

NFT의 `transferFrom(from = msg.sender, …)`이 from이 실제 주인인지 확인하고, 아니면 `ERC721IncorrectOwner`로 revert합니다.

Q6. listNFT가 safeTransferFrom 대신 transferFrom을 쓰는 이유는? (6-3장)

받는 쪽이 NFT를 다룰 줄 아는 AMM 자신이라 `onERC721Received` 확인이 필요 없습니다. safe를 쓰면 AMM이 그 함수를 구현해야 합니다.

Q7. cancelListing에서 `Listing memory`를 쓰면 어떤 사고가 나나? (7장)

복사본의 active만 false가 되고 원본은 true로 남아 "유령 판매 건"이 생깁니다. 바로 사면 NFT 이동에서 실패해 전체가 되돌아가지만, 판매자가 같은 NFT를 재등록하면 유령 건으로 그 NFT를 **옛 가격에** 사 갈 수 있습니다.

Q8. buyNFT의 대금은 AMM을 거치나? 그래서 무엇이 보장되나? (8장)

거치지 않고 구매자에서 판매자로 바로 갑니다. 풀의 reserve와 AMM 잔액이 바뀌지 않아 마켓이 풀과 독립입니다.

Q9. buyNFT에서 `active = false`를 외부 호출보다 먼저 하는 이유는? (8장)

CEI. 외부 호출 중 재진입해도 두 번째 호출은 `not active`로 막힙니다(nonReentrant와 이중 방어).

Q10. 구매에는 approve가 필요하고 취소에는 필요 없는 이유는? (11-3장)

구매는 **구매자의** 토큰을 AMM이 옮기고, 취소는 AMM이 **자기가 맡은** NFT를 옮기기 때문입니다.

Q11. `listings(id)`의 ABI 반환 순서를 바꾸면? (5-2장)

반환값은 순서로 해석되므로 필드가 엉뚱하게 읽힙니다. struct 순서(seller, nft, tokenId, price, paymentToken, active) 그대로 적어야 합니다.

Q12. ERC-20과 ERC-721의 Transfer 로그를 구분하는 법은? (9장)

topic0은 같고, ERC-721은 tokenId가 indexed라 topics가 4개이고 data가 비어 있습니다. ERC-20은 topics 3개, data에 value.

Q13. 컨트랙트에 이미지를 저장하지 않는 이유와, 그래도 바꿔치기할 수 없는 이유는? (4장)

storage가 너무 비싸서 주소만 저장합니다. CID는 내용의 해시라서 같은 CID가 다른 내용을 가리킬 수 없습니다.

Q14. 취소 후 같은 NFT를 다시 등록하면 판매 번호는? (5-3장)

새 번호(nextListingId 값)를 받습니다. 이전 건은 active=false로 기록이 남습니다.

Q15. 실제 구매 tx의 로그 3개를 순서대로 말해 보세요. (8장)

R2B Transfer(구매자 → 판매자, 50e18) → NFT Transfer(AMM → 구매자, #0) → NFTSold(0, 구매자, 판매자, 50e18).

APPENDIX

선택자·토픽 표, 한 장 요약

함수 선택자

함수	선택자	함수	선택자
`ownerOf(uint256)`	`0x6352211e`	`listNFT(address,uint256,uint256,address)`	`0xd68cdc52`
`getApproved(uint256)`	`0x081812fc`	`cancelListing(uint256)`	`0x305a67a8`
`approve(address,uint256)`	`0x095ea7b3`	`buyNFT(uint256)`	`0x51ed8288`
`setApprovalForAll(address,bool)`	`0xa22cb465`	`listings(uint256)`	`0xde74e57b`
`tokenURI(uint256)`	`0xc87b56dd`	`nextListingId()`	`0xaaccf1ec`

이벤트 topic0

이벤트	topic0
`Transfer(address,address,uint256)`	`0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef`
`Approval(address,address,uint256)`	`0x8c5be1e5ebec7d5bd14f71427d1e84f3dd0314c0f7b2291e5b200ac8c7c3b925`
`ApprovalForAll(address,address,bool)`	`0x17307eab39ab6107e8899845ad3d59bd9653f200f220920489ca2b5937696c31`
`NFTListed(…)`	`0x1e25fc8c928dd31a9c3eaae05546a0ccbbd49ce772d96d508940bd80708ade0c`
`NFTListingCancelled(uint256,address)`	`0x60d684fb8e0e3f4ae9b51912760d57e90172fe4eac3d2faed8f0b97fa6030cdb`
`NFTSold(uint256,address,address,uint256)`	`0x8562eb94748e30bb0071e932744ce902b460c683474fcb3bfd567ffa16b1afb2`

한 장 요약

주제	한 줄
ERC-721	"번호 → 주인" 장부. 번호마다 다른 토큰
허락	주인 본인 / `setApprovalForAll`(전부) / `approve(to, id)`(번호 하나). 이동하면 번호별 허락 삭제
TokenizeArt	owner만 `mint(to)`, 번호 0부터, 모든 번호가 같은 `tokenURI`
메타데이터	온체인엔 `ipfs://CID` 만, JSON과 png는 IPFS. 게이트웨이로 https 변환
에스크로	등록 시 NFT가 AMM으로, 판매/취소 시 구매자/판매자에게
Listing	(seller, nft, tokenId, price, paymentToken, active). getter는 이 순서의 튜플
listNFT	토큰 A/B만, price > 0, 저장 후 transferFrom, NFTListed
cancelListing	**storage** 참조, active와 판매자 확인, active=false, NFT 반환
buyNFT	active 확인 → false → 대금 구매자→판매자 → NFT AMM→구매자 → NFTSold
독립성	reserve 장부와 AMM 잔액을 건드리지 않음
프론트	ownerOf 스캔(allowFailure), useQuery로 메타데이터, 카드별 컴포넌트, 튜플 → 객체